

Chaînes de caractères

Exercice 1

Répondez aux questions suivantes en écrivant le code source respectif :

1. Déclaration d'une variable du type chaîne de caractères dénommée **firstName**.

.....

2. Déclaration d'un attribut du type chaîne de caractères dénommé **town**, initialisé avec la valeur « Luxembourg ».

.....

3. La variable **weightValue** (du type **String**) contient la valeur « 78 ». Il faut transformer cette valeur en un nombre entier et la sauvegarder dans la variable **weight** (du type **int**).

.....

4. La variable **age** (du type **double**) contient la valeur 16.6. Il faut transformer cette valeur en un **String** et la sauvegarder dans la variable **ageValue** (du type **String**).

.....

5. Notez l'expression pour tester si le contenu de la variable **blabla** (du type **String**) est égal à « vacances ».

.....

6. Sauvegarder la position de « pol » dans la chaîne de caractères **text** (du type **String**) dans la variable **posi** (du type **int**).

.....

7. Notez l'expression pour tester si la variable « comment » (du type **String**) contient le contenu de la variable « badWord » (du type **String**).

.....

8. Afficher le contenu des variables « wordA » et « wordB » (toutes les deux du type **String**) dans l'ordre lexicographique.

.....

.....

.....

Exercice 2

Développer la classe **StringOperations** qui contient un attribut **text** du type String sur lequel on peut appliquer les opérations suivantes :

reverse()

inverse l'ordre des caractères dans **text**.

isPalindrome()

retourne true ssi **text** est un palindrome. **text** peut contenir une phrase et contenir des lettres accentuées. Dans ce cas, les espaces et les symboles (; , : . etc.) sont ignorés et les lettres accentuées identifiées aux lettres non accentuées.

Les phrases suivantes sont considérées comme palindromes :

- "Panic in a Titanic, I nap."
- "Elu par cette crapule."
- "Engage le jeu que je le gagne."
- "La mère Gide digère mal."

Remarque : il est recommandé de développer des méthodes supplémentaires aidant à remplacer des symboles ou lettres accentuées.

getNumberOfWords()

retourne le nombre de mots dans **text**. On suppose que le texte peut contenir plusieurs espaces entre deux mots ainsi que des signes d'inter-ponction entourés d'espaces.

Exemple : La phrase suivante contient 37 mots :

"Palindrome : un texte ou un mot dont l'ordre des lettres reste le même qu'on le lise de gauche à droite ou de droite à gauche comme dans l'expression « Ésope reste ici et se repose. ». "

Exercice 3

Le mot «**cryptographie**» vient du grec *kryptos*, «caché», et *graphein*, «écrire». Il désigne l'ensemble des procédés permettant de transformer un message écrit, dit clair, en un autre message, dit chiffré ou crypté, par le moyen de certains codages convenus d'avance; le texte reçu n'est donc compréhensible que pour celui qui en connaît la clé.

Chiffage: Changer les données en une forme non compréhensible en appliquant un algorithme de chiffage.

Déchiffage: Reconstituer les données à leur forme initiale.

Le chiffre de César (Shift by 3)



La cryptographie existe déjà depuis longtemps. Le chiffage le plus connu (mais pas le plus ancien) est celui de Jules César.

César a crypté ses messages en employant une méthode qu'on a appelée ensuite le «Chiffre de César». Le chiffre de César consistait simplement à décaler les lettres de l'alphabet de quelques positions vers la droite ou la gauche.

On trouve une description du chiffre de César dans les **Vies des douze Césars** de **Suétone** (historien antique qui vivait autour de 100 après J.-C.) :

... Extant et ad Ciceronem, item ad familiares domesticis de rebus, in quibus, si qua occultius perferenda erant, per notas scripsit, id est sic structo litterarum ordine, ut nullum verbum effici posset: quae si qui investigare et persequi velit, quartam elementorum litteram, id est D pro A et perinde reliquas commutet. ...

(Livre premier, César, LVI)

... On a conservé en outre ses lettres à Cicéron, et celles qu'il adressait à ses familiers sur ses affaires domestiques; quand il avait à leur faire quelque communication secrète, il usait d'un chiffre, c'est-à-dire qu'il brouillait les lettres de telle façon qu'on ne pût reconstituer aucun mot: si l'on veut en découvrir le sens et les déchiffrer, il faut substituer à chaque lettre la troisième qui la suit dans l'alphabet, c'est-à-dire le D à l'A, et ainsi de suite. ...

L'empereur Auguste utilisait un chiffre similaire, mais il décalait les lettres d'un rang seulement. **Exemple:**

Décalons par exemple les lettres de 3 rangs vers la gauche, comme le faisait Jules César:

Clair	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Chiffré	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Ainsi, le message : AVE CAESAR MORITURI TE SALUTANT

devient : DYH FDHVDU PRULWXUL WH VDOXWDQW

Malgré (ou à cause de) sa simplicité, cet encryptage chiffre fut encore employé par des officiers sudistes pendant la guerre de Sécession et par l'armée russe en 1915.

Le chiffre ROT13, apparu en 1984 dans un programme permettant de lire les "News" de USENET, décale les lettres de 13 rangs.

Travail à réaliser:

Développer une classe **Crypter** qui permet de chiffrer et de déchiffrer des textes par la méthode '**Shift by N**' (c.-à-d. un déplacement des lettres de l'alphabet de **n** positions.).

Version pure: Convertir d'abord le texte à chiffrer en majuscules. Chiffrer/Déchiffrer uniquement les lettres majuscules de l'alphabet en effectuant une rotation de **n** positions entre les lettres de l'alphabet. Les autres caractères restent inchangés. Limitez **n** à une valeur entre 1 et 26.

Version élargie: Chiffrer/Déchiffrer tous les caractères en effectuant une rotation de **n** positions (**n** étant un entier quelconque).

Chiffre de Vigenère

Agé de vingt-six ans, **Blaise de Vigenère (1523-1596)**, diplomate français, passa deux années en mission diplomatique à Rome, où il se familiarisa avec les écrits sur la cryptographie de [Alberti](#), [Trithème](#) et [Porta](#). Au début, son intérêt pour la cryptographie était purement pratique et lié à son activité diplomatique. Une dizaine d'années plus tard, vers 1560, Vigenère considéra qu'il avait mis de côté assez d'argent pour abandonner sa carrière et se consacrer à l'étude. C'est seulement à ce moment-là qu'il examina en détail les idées de ses prédécesseurs, tramant grâce à elles un nouveau chiffre, cohérent et puissant. Bien qu'[Alberti](#), [Trithème](#), [Bellaso](#) et [Porta](#) en aient fourni les bases du chiffre, c'est du nom de Vigenère que ce nouveau chiffre fut baptisé, en l'honneur de l'homme qui lui donna sa forme finale.

Le **chiffre de Vigenère** est une amélioration décisive du [chiffre de César](#), trop facile à casser. Ce chiffre utilise une **clé** qui définit le décalage pour chaque lettre du message (A: décalage de 1 position, B: 2 positions, C: 3 positions, ..., Z: 26 positions).

Exemple: codons le texte "CHIFFRE DE VIGENERE" avec la clé "BACHELIER".

Clair	C	H	I	F	F	R	E	D	E	V	I	G	E	N	E	R	E
Clé	B	A	C	H	E	L	I	E	R	B	A	C	H	E	L	I	E
Décalage	2	1	3	8	5	12	9	5	18	2	1	3	8	5	12	9	5
Chiffré	E	I	L	N	K	D	N	I	W	X	J	J	M	S	Q	A	J

La grande force du chiffre de Vigenère est que la même lettre sera chiffrée de différentes manières. Par exemple le E du texte clair ci-dessus a été chiffré successivement M V L P I, ce qui rend inutilisable l'[analyse des fréquences](#) classique.

Travail à réaliser:

Copiez les méthodes développées dans la partie 2 et modifiez-les de façon à ce qu'elles permettent de chiffrer et de déchiffrer des messages par le chiffre de Vigenère. La clé n'est pas mémorisée dans la classe ! Elle peut contenir tous les caractères possibles. Toujours est-il qu'une lettre 'A' dans la clé doit provoquer une rotation d'une unité.

Informations supplémentaires: <http://www.ars-cryptographica.com>

Exceptions

Exercice 4 - Extraits du tutoriel Java de Oracle

Question 1:

Is the following code legal?

```
try {  
    ...  
} finally {  
    ...  
}
```

Question 2:

What exception types can be caught by the following handler?

```
catch (Exception e) {  
  
}
```

What is wrong with using this type of exception handler?

Question 3:

Is there anything wrong with this exception handler as written? Will this code compile?

```
try {  
  
} catch (Exception e) {  
  
} catch (ArithmeticException a) {  
  
}
```

Question 4:

Match each situation in the first list with an item in the second list.

- a) `ArrayList<String> sl; String s = sl.get(0);`
 - b) The JVM starts running your program, but the JVM can't find the Java platform classes. (The Java platform classes reside in `classes.zip` or `rt.jar`.)
 - c) A program is reading a stream and reaches the end of stream marker.
 - d) Before closing the stream and after reaching the end of stream marker, a program tries to read the stream again.
-
- 1. error
 - 2. checked exception
 - 3. compile error
 - 4. no exception

Exercice 5

Expérimentez avec les exceptions, et notez chaque fois vos constatations, les messages obtenus et vos conclusions :

- Créez une classe avec une méthode lançant une exception obligatoire (**checked exception**), p.ex. une `IOException`.
- Traitez les exceptions uniquement par **throws** sans utiliser **try catch**.
- Appelez la méthode critique par une autre méthode dans la même classe
- Appelez la méthode critique par une autre méthode dans une autre classe.
- Ajoutez une troisième classe appelant la méthode critique de la deuxième classe. Faites l'appel dans la méthode `main(...)`.
- Que remarquez-vous si la classe 'supérieure' (dans la hiérarchie des appels) ne traite pas l'exception, mais effectue aussi un **throws** ?
- Traitez les exceptions par **try catch** aux différents niveaux. Enlevez les instruction **throws** superflues. Faites afficher un '**stack trace**' dans le cas d'une erreur.
- Refaites le même exercice avec une exception du type `RuntimeException`. (p.ex. `ArithmeticException`).
- Entourez l'instruction critique et les appels des méthodes critiques par des affichages p.ex : `"callCriticalMethod2: before call"`, `"callCriticalMethod2: after call"`. Ajoutez un affichage à la fin des méthodes. Faites des essais en plaçant **try-catch** aux différents niveaux (dans les différentes méthodes). Que constatez-vous ?
- Ajoutez des blocs **try ... finally** à toutes les méthodes, désactivez les blocs **catch** et observez la suite des traitements.
- Est-ce que les deux blocs suivants réagissent de la même façon ? Motivez votre réponse !

```
(A)    try {  
        <bloc d'instructions 1>  
    }  
    catch (...) {  
        <bloc d'instructions 2>  
    }  
    finally {  
        <bloc d'instructions 3>  
    }
```

```
(B)    try {  
        <bloc d'instructions 1>  
    }  
    catch (...) {  
        <bloc d'instructions 2>  
    }  
    <bloc d'instructions 3>
```

Exercice 6

Même si dans le cours, nous avons défini une exception du type **DivisionByZeroException**, pour le cas où le dénominateur est zéro, ce n'est pas la meilleure manière de procéder, car dans ce cas c'est trop tard : la fraction a déjà été initialisée avec la valeur indésirable.

Mieux vaut éviter ce cas dès le départ :

1. Reprenez votre classe **Fraction** et lancez une exception du type **IllegalArgumentException** dès qu'on essaie d'affecter une valeur nulle au dénominateur. Enlevez tous les autres tests sur une valeur nulle du dénominateur. Affichez un message d'erreur informatif.
2. Testez l'exception, en Unimozer.
3. Développez un programme (en Unimozer ou en NetBeans) pour tester votre classe. Interceptez l'erreur et affichez son message à l'écran.
4. Après avoir ajouté une méthode **divideBy(Fraction)** à votre classe, un autre cas de figure se présente : Définissez un nouveau type d'exception **DivisionByZeroException** (dérivé de **ArithmeticException**) que vous provoquez dans ce cas.
5. Testez les deux exceptions, en Unimozer, puis développez une application pour tester toutes les opérations et les exceptions qu'elles peuvent provoquer. Interceptez les deux exceptions avec deux blocs **try ... catch**.

Exercice 7

Reprenez la classe **Crypter**:

1. Déterminez les cas où il peut s'avérer nécessaire de produire une exception. Testez votre classe pour ce cas. Quel type d'exception est approprié ?
2. Définissez un type d'exception individualisé pour ce type d'erreur.
3. Testez les exceptions avec une classe qui intercepte l'exception et réagit en conséquence.
4. Définissez un message par défaut pour le nouveau type d'exception.

Tableaux et Listes

Exercice 8 - Tableaux

Développez une classe qui mémorise les données d'une personne : nom, prénom, adresse (no, rue), adresse (cp & cité)

Créez une classe **Persons** qui sait mémoriser et gérer une liste de personnes à l'aide d'un tableau (**array**).



Exercice 9 - ArrayList

Copiez le projet de l'exercice ci-dessus et réalisez la même chose en employant une liste du type **ArrayList** au lieu d'un tableau.

Exercice 10 - JList

Ouvrez votre projet de l'exercice précédant (**ArrayList**) en NetBeans et créez une interface graphique pour entrer les données pour chaque personne. Ajoutez les données dans la liste **Persons**. Veillez à ce que la classe **Person** possède une méthode **toString** affichant les données de la personne. Faites afficher la liste actuelle des personnes dans une liste du type **JList**. Que se passe-t-il si vous enlevez (mettez en commentaires) la méthode **toString** de **Person** ?

Traitement de fichiers

Exercice 11 - Fractions

Reprenez le code de l'exercice **Fraction**. Ajoutez une classe **Fractions** qui sait gérer une liste (**ArrayList**) **alFractions** de fractions.

Ajoutez ensuite les méthodes suivantes à la classe **Fractions** :

<code>public void saveToDataFile(String fileName)</code>	sauvegarde dans un fichier de <u>données</u>
<code>public void loadFromDataFile(String fileName)</code>	lecture dans un fichier de <u>données</u>
<code>public void saveToTextFile(String fileName)</code>	sauvegarde dans un fichier <u>texte</u>
<code>public void loadFromTextFile(String fileName)</code>	lecture dans un fichier <u>texte</u>
<code>public void saveToObjectFile(String fileName)</code>	sauvegarde dans un fichier <u>d'objets</u>
<code>public void loadFromObjectFile(String fileName)</code>	lecture dans un fichier <u>d'objets</u>

Examinez le contenu des fichiers à l'aide d'un éditeur de textes.

Exercice 12 - Persons

Reprenez le code de l'exercice **Persons** (*array*) et ajoutez les attributs suivants à la classe **Person** :

Height	:	type int (en cm)
Weight	:	type double (en kg)

Ajoutez ensuite les méthodes suivantes à la classe **Persons** :

<code>public void saveToDataFile(String fileName)</code>	sauvegarde dans un fichier de <u>données</u>
<code>public void loadFromDataFile(String fileName)</code>	lecture dans un fichier de <u>données</u>
<code>public void saveToTextFile(String fileName)</code>	sauvegarde dans un fichier <u>texte</u>
<code>public void loadFromTextFile(String fileName)</code>	lecture dans un fichier <u>texte</u>
<code>public void saveToObjectFile(String fileName)</code>	sauvegarde dans un fichier <u>d'objets</u>
<code>public void loadFromObjectFile(String fileName)</code>	lecture dans un fichier <u>d'objets</u>

Examinez le contenu des fichiers à l'aide d'un éditeur de textes.

Exercice 13 - Vigenère

Reprenez le projet **Coder** (Vigenère) et ajoutez une possibilité pour encrypter et décrypter des fichiers binaires octet par octet selon la méthode de Vigenère.

Les principes de la POO

Exercice 14 - Modélisation d'adresses

En vue du développement d'une application de gestion d'adresses, modélisez les classes **Entity**, **Person** et **Organisation** en profitant au mieux des concepts de la POO :

1. On peut ajouter au carnet d'adresses (qui n'est pas à développer!) n'importe quelle entité. Une entité est soit une personne physique, soit une organisation.
2. Toute entité possède une méthode **getAddress()** qui retourne son adresse.

- a) L'adresse d'une personne physique a toujours la forme suivante :

```
<prenom> <nom>
<n°, rue / bp>
<cp, ville>
```

- b) Une organisation peut posséder une personne de contact. Dépendant de ceci, son adresse peut prendre l'une des formes suivantes :

```
<nom de l'organisation>          <nom de l'organisation>
c/o <contact prénom> <contact nom>  <n°, rue / bp>
<n°, rue / bp>                  <cp, ville>
<cp, ville>
```

The diagram illustrates the modeling of addresses for two classes: **Person** and **Organisation**.

Scenario 1: Person

- Class Data:**

```
p1 : Person
firstname = Pit
name = Weyer
adrBpOrStreet = 43, rue Bellevue
adrCpAndTown = L-7642 Muppenduerf
```
- Result Window:**

```
Result
Pit Weyer
43, rue Bellevue
L-7642 Muppenduerf
OK
```

Scenario 2: Organisation

- Class Data:**

```
org1 : Organisation
contact = <NULL>
name = LTAM
adrBpOrStreet = 19, rue Guillaume Schneider
adrCpAndTown = L-2522 Luxembourg
```
- Result Window:**

```
Result
LTAM
19, rue Guillaume Schneider
L-2522 Luxembourg
OK
```

Scenario 3: Organisation with Contact Person

- Class Data:**

```
p1 : Person
firstname = Pit
name = Weyer
adrBpOrStreet = 43, rue Bellevue
adrCpAndTown = L-7642 Muppenduerf

org1 : Organisation
contact = Person@759a2097
name = LTAM
adrBpOrStreet = 19, rue Guillaume Schneider
adrCpAndTown = L-2522 Luxembourg
```
- Result Window:**

```
Result
LTAM
c/o Pit Weyer
19, rue Guillaume Schneider
L-2522 Luxembourg
OK
```

Exercice 15 - Polymorphisme et tableaux

[Familiarisez-vous d'abord avec le chapitre *Array* du cours "*Les structures de données*".]

Ajoutez au projet une classe **AddressBook** qui contient comme attribut principal **addresses** un tableau d'entités (de l'exercice précédent).

Ajoutez les méthodes et attributs nécessaires pour pouvoir ajouter des adresses d'organisations et de personnes à un carnet d'adresses. La gestion des dimensions (maximale et effective) doit être transparente pour l'utilisateur.

La méthode **toString** retourne le nombre d'adresses ainsi qu'une liste de tous les noms des destinataires contenus dans la liste.

La méthode **printAll** affiche ligne par ligne toutes les adresses des entités dans la fenêtre des messages.

Profitez au maximum de l'héritage et du polymorphisme.

Pour faciliter les tests, vous pouvez ajouter une classe **AddressBookController** qui remplit votre carnet d'adresses avec quelques données et affiche les résultats.

Exercice 16 - Modélisation & réalisation

Le magasin **Neptun** vend toutes sortes de média (livres, CD, DVD, disques BluRay, jeux PC, PS3). Chaque type de médium possède des informations individuelles. P.ex. les livres possèdent un numéro ISBN ; les DVD et BD contiennent des informations sur le régisseur et la région, les DVD, BD et jeux possèdent des informations FSK, . . .

Les clients possèdent une carte client (évidemment leurs données personnelles sont mémorisées). Les employés obtiennent un revenu mensuel régulier plus un supplément de 1 pour-cent de la somme qu'ils ont vendus (sans TVA).

Chaque objet possède un constructeur qui initialise tous ces attributs.

Chaque objet possède une méthode **toString** qui retourne toutes ses informations formatées sur plusieurs lignes.

Un client peut acheter un article à un employé.

OOA (Object Oriented Analysis) :

Analysez le petit univers décrit ci-dessus et déterminez les classes nécessaires. Déterminez les actions qui doivent être possibles et les informations qui doivent être stockées. Déterminez les relations entre les différentes classes. Représentez vos résultats avec des diagrammes de classes en UML.

OOD (Object Oriented Design) :

En tenant compte du langage de programmation, précisez le modèle des classes issu de votre analyse et complétez-le avec les attributs et méthodes nécessaires. Simplifiez votre modèle en profitant au maximum de l'héritage et du polymorphisme. Si nécessaire, ajoutez des classes pour la gestion des objets. Si vous voulez programmer une interface utilisateur alors respectez le modèle MVC.

OOP (Object Oriented Programming) :

Réalisez les classes issues de votre conception en Java. Testez votre application.

Au cours chacune des étapes il se peut qu'il soit nécessaire de retravailler les étapes précédentes pour remédier à d'éventuels défauts ou imperfections.

[Familiarisez-vous d'abord avec le chapitre *ArrayList* du cours "Les structures de données".]

Exercice 17 - AddressBook & ArrayList

Reprenez l'exercice **AddressBook** et remplacez le tableau statique par un tableau dynamique du type **ArrayList**.

Exercice 18 - Neptun & ArrayList

Reprenez l'exercice **Neptun** et remplacez les tableaux statiques par des tableaux dynamiques du type **ArrayList**.

[Familiarisez-vous d'abord avec le développement d'interfaces graphiques simples et plus spécialement avec le chapitre *JList* du cours "Les structures de données".]

Exercice 19 - AddressBook & GUI

Reprenez l'exercice **AddressBook** et développez un interface graphique simple en employant la représentation des données dans une liste du type **JList**.

Exercice 20 - Neptun & GUI

Reprenez l'exercice **Neptun** et développez un interface graphique simple en employant la représentation des données dans des listes du type **JList**.

Exercice 21 - Interfaces

- Réalisez les classes **Shape**, **Circle**, **Square**, **Rectangle**, **Triangle**. Chaque figure a une position (point supérieur gauche : x,y) et une couleur. Ces attributs et les dimensions (rayons, longueurs de côtés) sont fixés lors de la construction.
- Chaque figure peut afficher son état avec **toString**.
- Toutes les figures implémentent l'interface **Computable** qui possède les méthodes **getSurface()** et **getPerimeter()**.
- Toutes les figures implémentent l'interface **Comparable** basé sur leur surface.
- Réalisez une classe **ShapeList1** qui sait gérer un ensemble de figures (**Shape**) sous forme d'un **array** statique. Les figures dans la liste sont toujours triées. La liste **ShapeList1** se laisse parcourir à l'aide d'un itérateur.
- Réalisez la classe **ShapeList2** qui correspond à **ShapeList1**, mais qui est basé sur une **ArrayList**.

"Loi de Hofstadter: Il faut toujours plus de temps que prévu, même en tenant compte de la loi de Hofstadter."

Exercice 22

- Réalisez la méthode récursive **power** qui calcule X^N pour un réel X et un entier N positif ou zéro. Il n'est pas nécessaire de définir un traitement spécial pour les cas où X^N n'est pas défini.
- Complétez la méthode récursive **power** pour calculer X^N pour un réel X et un entier N positif, négatif ou zéro.

Exercice 23

Réalisez la méthode récursive **gcd** qui calcule le plus grand commun diviseur de deux entiers x et y selon la définition suivante:

$$\begin{cases} \text{gcd}(x, 0) = x \\ \text{gcd}(x, y) = \text{gcd}(y, x \% y) \end{cases} \quad \text{pour } y > 0$$

Exercice 24

Réalisez cet exercice sur papier, sans le programmer !

Soit la méthode suivante :

```
public boolean machin(int x)
{
    if (x==0) return true;
    else     return !machin(x-1);
}
```

- Indiquez la suite des appels récursifs et le résultat pour l'appel **machin(3)**.
- A quoi sert la méthode **machin** ?
- Pour quelles valeurs de X la méthode **machin** fournit-elle un résultat ?

Soit la méthode suivante :

```
public double truc( double x, int y)
{
    if      (y==0)    return 1;
    else if (y%2==0) return truc(Math.pow(x,2), y/2);
    else           return truc(x, y-1) * x;
}
```

- Indiquez la suite des appels récursifs et le résultat pour l'appel **truc(2,5)** ?
- A quoi sert la méthode **truc** ?

"Cette phrase est sans signification puisqu'elle se réfère à elle-même"

Exercice 25

L'origine des nombres de **Fibonacci** remonte à 1202 à Leonardo de Pisa, (fils de Bonacci – "*filius Bonacci*") et repose sur un principe simple: évaluer à quelle vitesse des lapins peuvent se produire dans des circonstances idéales. Prenez un couple de lapins qui vient de naître, supposez que ces lapins peuvent procréer dès l'âge d'un mois et qu'il en résulte chaque fois un autre couple de lapins.

Au début du troisième mois, vous aurez deux couples de lapins. Au début du quatrième mois, le couple initial engendre un autre couple de lapins, ce qui donne un total de trois couples. Au cours du quatrième mois, le deuxième couple se met également à procréer, ce qui donne cinq couples au début du cinquième mois. Même si les lapins ont démarré lentement, ils rattrapent vite le temps perdu. Imaginez, au bout de 25 ans à ce rythme, il y aurait

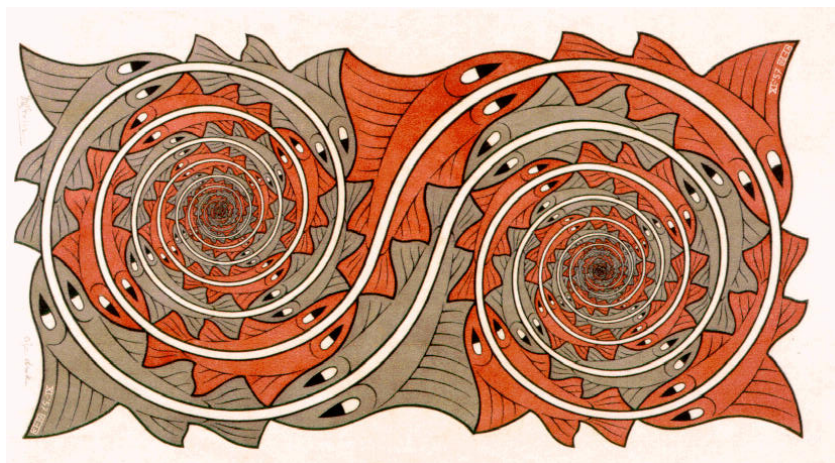
222 232 244 629 420 445 529 739 893 461 909 967 206 666 939 096 499 764 990 979 600

couples de lapins!

Version moderne du même problème:

Imaginez que tous les utilisateurs d'un programme (*freeware* – bien sûr) agissent comme suit: Le premier jour, ils testent le programme et à partir du deuxième jour, ils passent chaque jour une copie du programme à un de leurs amis qui agit alors de la même façon ...

- Réalisez la méthode récursive **fibo_r** qui calcule le nombre de couples de lapins (de copies du programme) après **n** mois (jours)!
- Réalisez la méthode itérative **fibo_i** qui calcule le nombre de couples de lapins (de copies du programme) après **n** mois (jours)!
- Testez et comparez les deux méthodes **fibo_r** et **fibo_i**!
- Représentez graphiquement (arbre) la suite des appels récursifs pour **fibo_r(6)**!
- Déterminez le nombre d'appels récursifs nécessaires pour calculer **fibo_r(1)**, **fibo_r(2)**, ..., **fibo_r(6)**!
Trouvez une formule générale pour déterminer le nombre d'appels récursifs qui sont générés au cours du calcul de **fibo_r(n)**!
Exprimez la formule sous forme d'une fonction **nar(n)** (nombre d'appels récursifs)!
- Essayez de trouver une formule qui décrit la relation entre **fibo_r(n)** et **nar(n)**!



M. C. Escher

Exercice 26 - suites et fonctions récursives

La suite de Newton permet de calculer la racine carrée d'un nombre **a** réel positif. Elle est définie par la relation suivante :

$$\begin{cases} u_0 = a \\ u_{n+1} = \frac{1}{2} \left(u_n + \frac{a}{u_n} \right) \end{cases}$$

Ecrire la méthode récursive **newton** qui retourne comme résultat le **n**^{ième} terme de la suite de **newton** pour un réel **a** donné. **Limitez le temps de calcul** au strict nécessaire !

Exercice 27 - séries de Taylor

Les développements en série de Taylor des fonctions *exp*, *cosinus* et *sinus* de la variable réelle *x* peuvent s'écrire :

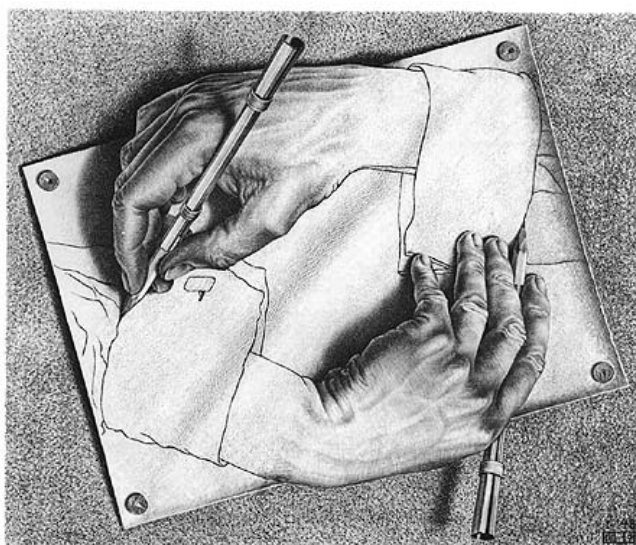
$$e^x = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots = \sum_{n=0}^{\infty} \left(\frac{x^{4n}}{(4n)!} - \frac{x^{4n+2}}{(4n+2)!} \right)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=0}^{\infty} \left(\frac{x^{4n+1}}{(4n+1)!} - \frac{x^{4n+3}}{(4n+3)!} \right)$$

Pour chacune des séries ci-dessus, écrire une méthode récursive qui effectue le développement de la série jusqu'au terme **n** (pour un réel **x** et un entier positif **n** donnés).

"La phrase que j'écris maintenant est la phrase que vous lisez maintenant."



Exercice 28

Pour les cas suivants, éliminez la boucle en effectuant les mêmes opérations à l'aide d'un sous-programme récursif! Indiquez aussi les parties qui font appel aux sous-programmes récursifs!

a)

```
for (int i=1 ; i<=10 ; i++)
    System.out.println(i);
```

En général:

```
for (int i=1 ; i<=n ; i++)
    <bloc d'instruction>
```

b)

```
for (int i=0 ; i<=3 ; i++)
    for (int j=0 ; j<=9 ; j++)
        System.out.println(i * 10 + j);
```

En général:

```
for (int i=0 ; i<=n ; i++)
    for (int j=0 ; j<=m ; j++)
        <bloc d'instruction>
```

Exercice 29

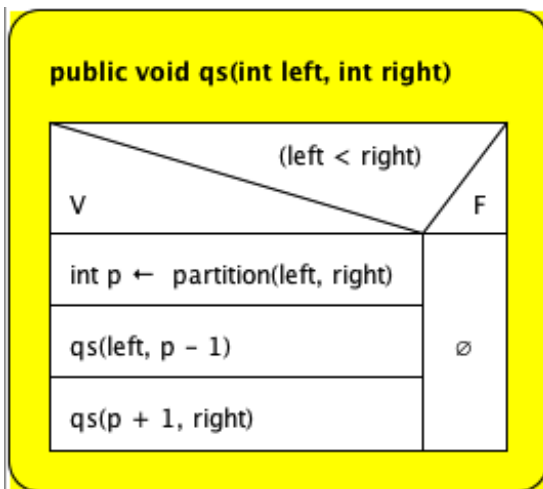
1. Développez d'abord une classe qui contient un tableau statique **tab** qui peut être rempli de nombres entiers aléatoires. **toString** retourne la liste des nombres, **printAll** les affiche dans la fenêtre des messages.
2. Ajoutez une méthode (non récursive) **selSort** pour trier les nombres selon la méthode du tri par sélection.
3. Ajoutez une méthode récursive **linearSearch** qui recherche la position d'un nombre dans la liste en traversant le tableau élément par élément. La méthode retourne comme résultat la position du nombre dans la liste ou bien la valeur -1 si la valeur ne se trouve pas dans la liste. Déterminez le temps de recherche en employant **System.currentTimeMillis()**.
4. Développez une méthode récursive **binarySearch** qui recherche la position d'un nombre dans un **tableau trié** en utilisant le procédé de la **recherche dichotomique** ! La méthode retourne comme résultat la position du nombre dans la liste ou bien la valeur -1 si la valeur ne se trouve pas dans la liste. Déterminez le temps de recherche en employant **System.currentTimeMillis()**.
5. Comparez le temps de recherche des deux algorithmes pour des tableaux de différentes grandeurs et établissez un graphique.

Exercice 30

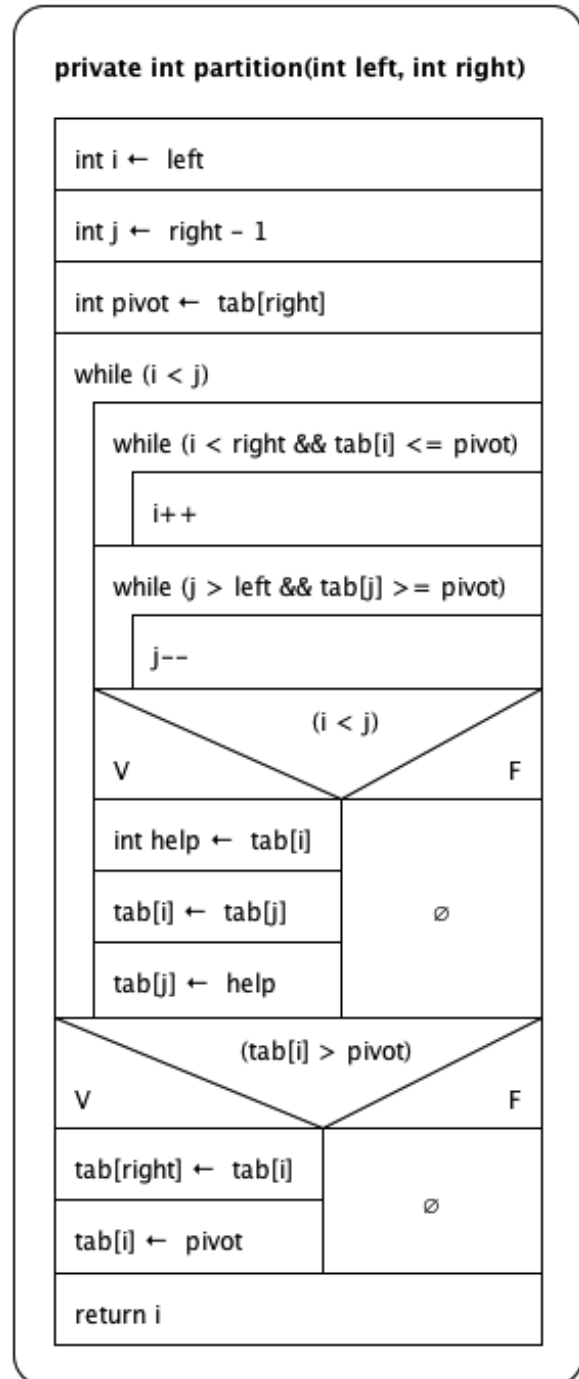
Soit un tableau **tab** contenant des nombres entiers et soient deux nombres entiers **left** et **right** tels que : $0 \leq \text{left} \leq \text{right} \leq \text{tab.length}-1$

- 1) En analysant le structogramme, déterminez l'effet de la méthode **partition** sur le contenu de **tab** et déterminez le résultat retourné !

Considérez ensuite la méthode récursive **qs** définie comme suit:



- 2) En analysant le structogramme, déterminez l'effet de la méthode **qs** sur le contenu d'une liste **tab** lors d'un appel par **qs(0, tab.length-1)**
- 3) Intégrez la méthode dans une classe contenant l'attribut **tab** pour tester la procédure **qs** (et **partition**).
- 4) Dans le même programme, réalisez la méthode non récursive **s1** qui a le même effet que **qs**, et qui a déjà été traitée auparavant.
- 5) Comparez la vitesse de travail des deux méthode **qs** et **s1** pour des listes de différentes grandeurs!



Exercice 31 - Les tours de Hanoï

La légende des Tours de Hanoï remonte aux origines des temps :

"Dans le grand et fameux Temple de Benares, sous le dôme marquant le centre du monde, trône un socle de bronze sur lequel sont fixées trois aiguilles de diamant, chacune d'elles haute d'une coudée et fine comme la taille d'une quèpe.

Sur une de ces aiguilles, à la Création, Dieu empila soixante quatre disques d'or pur, du plus grand au plus petit, le plus large reposant sur le socle de bronze.

Il s'agit de la tour de Bramah. Jour et nuit, inlassablement, les moines déplacent les disques d'une aiguille vers l'autre tout en respectant les immuables lois de Bramah qui obligent les moines à ne déplacer qu'un disque à la fois et à ne jamais le déposer sur un disque plus petit.

*Quand les soixante quatre disques auront été déplacés de l'aiguille sur laquelle Dieu les déposa à la Création vers une des autres aiguilles, la tour, le temple, et les Brahmanes seront réduits en poussière, et le monde disparaîtra dans un grondement de tonnerre."*¹



En résumé: On a 3 aiguilles en face de soi, numérotées 1, 2 et 3 de la gauche vers la droite, et n disques de tailles toutes différentes entourant l'aiguille 1, formant un cône avec le plus gros en bas et le plus petit en haut. On veut amener tous les disques de l'aiguille 1 à l'aiguille 3 en ne prenant qu'un seul disque à la fois, et en s'arrangeant pour qu'à tout moment il n'y ait jamais un disque sous un disque plus gros. La légende dit que les moines passaient leur vie à Hanoi à résoudre ce problème pour $n=64$.

Ce problème compliqué à première vue peut être résolu à l'aide d'un algorithme récursif de quelques lignes seulement.

Familiarisez-vous avec le problème posé en essayant de le résoudre (sans ordinateur) pour un nombre limité de disques (p.ex. pour $n=4$). Essayez de généraliser votre méthode de résolution. Déduisez-en une méthode récursive **hanoi** dont l'en-tête est définie comme suit:

```
public void hanoi (int n, int i, int j)
```

avec: **n** nombre de disques à déplacer
i numéro de l'aiguille de départ
j numéro de l'aiguille d'arrivée

Exemple d'appel: **hanoi (64,1,3)** affiche tous les déplacements à faire pour déplacer 64 disques de l'aiguille 1 vers l'aiguille 3.

P.ex: la ligne '1->2' décrit le déplacement d'un disque de l'aiguille 1 vers l'aiguille 2.

Combien de déplacements faut-il faire pour résoudre le problème pour n disques? Démontrez votre hypothèse! En supposant que le déplacement d'un disque d'or prend une seconde, combien de temps mettront les moines pour provoquer **la fin du monde** ?

¹ En réalité cette légende fut inventée par le mathématicien français E. Lucas en 1883

Exercice 32 - Sauvegarde/Lecture de données - format binaire

Recherchez de l'aide sur les mots clés suivants du paquet `java.io` :

`Serializable, FileOutputStream, FileInputStream, ObjectOutputStream, ObjectInputStream, IOException, writeObject, readObject, close`

Copiez l'exercice '`ShapeList`' et ajoutez la possibilité de sauvegarder la liste et toutes les figures sous forme binaire et de les rétablir.

Exercice 33 - Sauvegarde/Lecture de données - format texte

Recherchez de l'aide sur les mots clés suivants du paquet `java.io` :

`FileReader, FileWriter, IOException, write, read, close`

Recherchez de l'aide sur `java.util.Scanner` et ses méthodes.

Copiez l'exercice '`ShapeList`' et ajoutez la possibilité de sauvegarder la liste et toutes les figures sous forme de texte et de les rétablir.

Conseil :

Mémoriser les données des figures ligne par ligne, en écrivant une ligne pour chaque figure (p.ex. à l'aide d'une nouvelle méthode `toFileString`). Il peut aussi être utile, d'écrire dans `Shape` une méthode statique (p.ex. `newFromString`) qui sait créer une figure à partir d'une ligne de texte.

Essayez deux variantes pour lire une ligne de texte dans un fichier (à l'aide de `read` et à l'aide de `Scanner`). Une exception est levée, si le format n'est pas correct.

Exercice 34 - Linked List

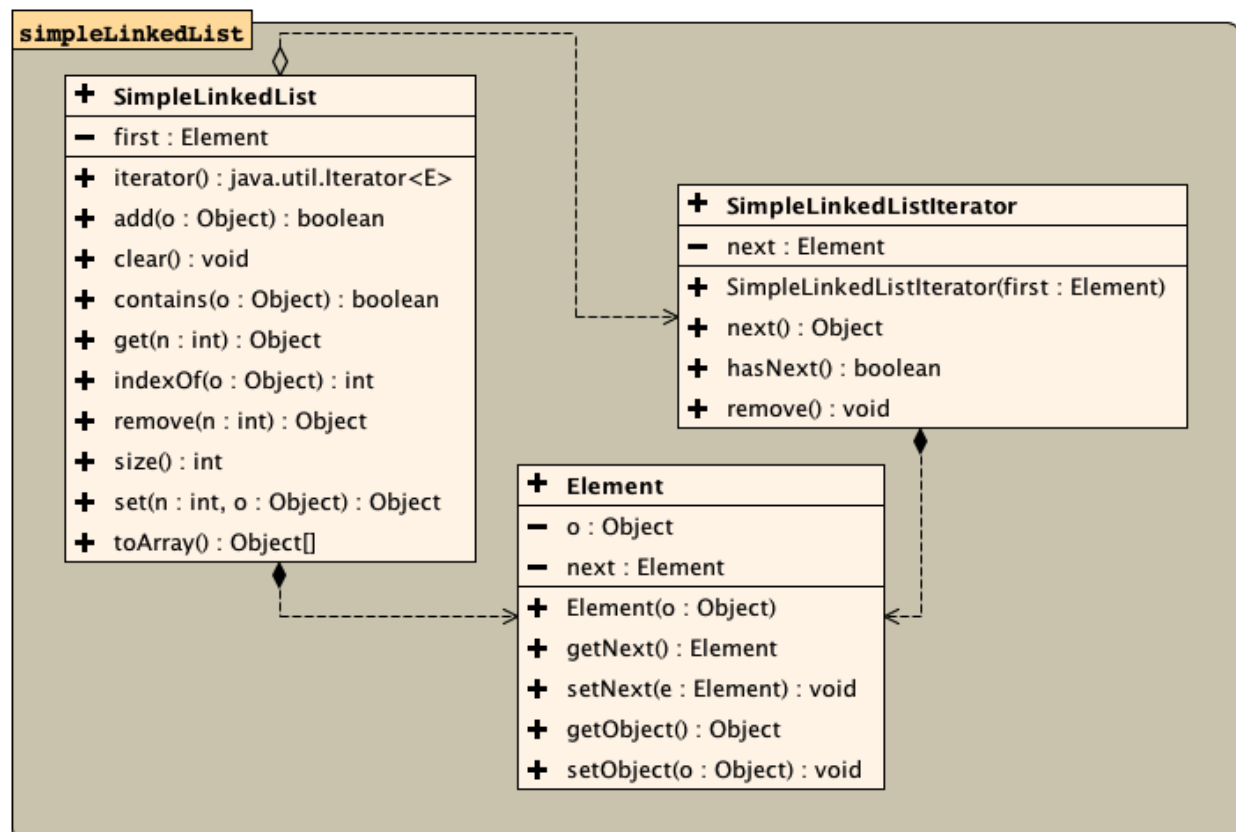
Dans la suite, vous allez programmer votre propre collection qui fonctionne selon le principe d'une **liste chaînée** (EN : **linked list**). La classe s'appellera **SimpleLinkedList** (pour la distinguer de la classe prédéfinie **LinkedList**).

Dans un nouveau paquet **simpleLinkedList**, réalisez les classes suivantes :

- Réalisez d'abord la classe **Element** avec un constructeur, ses accesseurs et ses manipulateurs pour ses deux attributs.
- Réalisez ensuite la classe **SimpleLinkedList** avec les méthodes du schéma UML. Les méthodes réalisées fonctionnent de la même façon que celles des autres collections (p.ex. **ArrayList**). Cependant, nous n'allons pas implémenter l'interface intégral **Collection**, mais seulement un petit sous-ensemble de ses méthodes. (→ voir **JavaDoc** pour les détails). Comme pour les autres collections, il est possible de spécifier un type pour les éléments de notre collection en l'indiquant comme paramètre lors de la définition (**types paramétrés** EN: **generics** → http://en.wikipedia.org/wiki/Generics_in_Java).

P.ex.: `private SimpleLinkedList<Person> shapes = new SimpleLinkedList<Person>();`

- Réalisez enfin un itérateur **SimpleLinkedListIterator** pour la classe **SimpleLinkedList**. Il n'est pas nécessaire de réaliser la méthode optionnelle **remove** (un appel de la méthode **remove** lance une exception **UnsupportedOperationException**).



Reprenez l'exercice **ShapeList2** (avec la gestion dans une **ArrayList** et la sauvegarde sous forme de texte). Après avoir intégré le paquet **simpleLinkedList** dans le projet, remplacez **ArrayList** par votre **SimpleLinkedList**. En principe le projet devrait fonctionner sans d'autres modifications avec votre nouvelle liste chaînée.

Exercice 35 - Les arbres binaires de recherche - Binary Search Tree

- Développez les classes **Node<E>** et **BST<E>** qui réalisent un arbre binaire de recherche. Implémentez pour **BST** les mêmes méthodes suivantes :
 - un constructeur
 - **add(E e)** ajoute l'objet/élément **e** à l'arbre (en gardant l'ordre interne)
 - **isEmpty()** retourne **true** si l'arbre est vide (voir remarques ci-dessous)
 - **clear()** efface l'arbre (voir remarques ci-dessous)
 - **contains(E e)** retourne **true** si l'élément fourni se trouve dans l'arbre
 - **getDepth()** retourne la profondeur de l'arbre.
 - **size()** retourne le nombre de noeuds de l'arbre (0 si vide)
 - **toArray()** les éléments de l'arbre dans un tableau (triés)
 - **indexOf(E e)** retourne l'indice de l'élément fourni comme paramètre
 - **get(int i)** retourne l'élément à l'indice **i** (ou **null**)
 - **remove(E e)** supprime l'élément de l'arbre et retourne **true** si réussi
 - **remove(int i)** supprime l'élément à l'indice **i** de l'arbre et retourne cet objet comme résultat (ou **null** si non réussi)

Le mot **indice** correspond à la position d'un élément dans la liste qu'on obtient si si on aligne les éléments par l'ordre interne de l'arbre (c.-à-d. par ordre croissant selon **compareTo**).

Lors de la réalisation de ces méthodes, il faut programmer quelques méthodes (privées) supplémentaires pour les opérations qui nécessitent une solution récursive.

- Développez un itérateur pour la classe **BST**.
- Testez les classes avec le projet **ShapeList**.

Remarques :

- Les noeuds doivent implémenter **Comparable**. Pour cette raison, aussi les objets **<E>** transportés par les noeuds doivent implémenter **Comparable**. Pour assurer que **E** implémente l'interface **Comparable** il faut définir **Node** comme :

Node<E extends Comparable<E>>

(Il est impossible de définir **<E implements Comparable<E>>**, mais **extends** peut être appliqué sur des interfaces lors de la définition de génériques.)

- La même remarque s'applique à **BST**.
- Comme il est impossible de passer des paramètres par référence, la racine de l'arbre ne peut pas être changée par une méthode. Ainsi, il faut définir un arbre vide comme suit (et il faudra en tenir compte dans toutes les méthodes):

Un arbre **vide** est un noeud où tous les attributs (objet, **left** et **right**) sont **null**.

Exercice 36 - Collections & Maps

- a) Etablir l'arbre d'héritage des interfaces suivants : **Iterable**, **Collection**, **List**, **Map**, **Set**, **SortedMap**, **SortedSet**, **NavigableMap**, **NavigableSet**.
- b) Etablir l'arbre d'héritage des collections prédéfinies **AbstractCollection**, **AbstractList**, **AbstractSet**, **TreeSet**, **ArrayList**, **AbstractSequentialList**, **LinkedList**, **Vector**, **Dictionary**, **Hashtable**, **AbstractMap**, **HashMap**, **TreeMap**.
- c) Qu'est-ce qu'une **Collection**? Quels sont ses caractéristiques ?
- d) Qu'est-ce qu'un **Set** ? Quels sont ses caractéristiques (en comparant avec **Collection**) ?
- e) Qu'est-ce qu'une **List**? Quels sont ses caractéristiques ?
- f) Qu'est-ce qu'une **Map** ? Quels sont ses caractéristiques et champs d'application ?
- g) Qu'est-ce qu'une **HashMap** ? Quels sont ses champs d'application ?
- h) Quelles sont les caractéristiques des interfaces **SortedMap** et **SortedSet** ?
- i) Quelles sont les caractéristiques de **TreeMap** et **TreeSet** ?
- j) Quelles sont les différences entre **ArrayList** et **Vector** ?
- k) Quelles sont les différences entre **HashMap** et **Hashtable** ?
- l) Que peut-on dire de **Vector** et **Hashtable** et de leur utilisation ?
- m) Qu'est-ce que la **synchronisation** dans le contexte des collections ? Quand est-ce qu'elle joue un rôle ?
- n) Comment peut-on **synchroniser** une collection qui ne l'est pas dès le départ ?